

Infraestrutura de Implantação

Finalidade

Este documento descreve a infraestrutura física e lógica necessária para executar o sistema Wiigu em um ambiente de demonstração local. Além disso, apresenta uma projeção dos requisitos de infraestrutura necessários caso a aplicação fosse implantada em um ambiente de produção real.

Ambiente de Demonstração

O protótipo foi construído como uma aplicação web dividida em três frentes: *frontend* interativo (**React**), API de serviços (**Express**) e banco de dados relacional embarcado (**SQLite**). Em cenário de demonstração acadêmica local, a aplicação é executada por meio dos seguintes serviços interconectados:

- O *frontend* é servido pela ferramenta de *build* **Vite** no endereço estático `http://127.0.0.1:5173`.
- A API (*backend*) roda via **Express** respondendo às requisições na porta `http://127.0.0.1:3001`.
- A persistência é realizada em um banco **SQLite** armazenado localmente no arquivo `prototype/data/wiigu.db`.

Hardware Necessário

Para demonstração (Avaliação):

- Computador ou *notebook* com processador moderno (arquitetura x64 ou ARM).
- Mínimo de 4 GB de memória RAM.
- Espaço em disco suficiente (cerca de 500 MB) para alocação do código-fonte, das dependências do Node.js (`node_modules`) e do arquivo gerado pelo SQLite.
- Tela de exibição de resolução padrão, teclado e *mouse* (ou *touchpad*) para as interações de *drag-and-drop*.

Para produção (Cenário Hipotético):

- Servidor (VPS ou Nuvem) com instâncias computacionais suficientes para executar os processos do Node.js.

- Armazenamento persistente e em bloco (SSD) para o banco de dados.
- Capacidade de replicação e *backup* programado do banco de dados.
- Link de rede (banda) estável com proteção básica contra ataques (ex: *firewall* / DDoS).

Software Necessário

Para demonstração:

- Sistema Operacional: Windows, Linux ou macOS.
- **Node.js** na versão 24 ou superior (que inclui suporte nativo ao módulo `node:sqlite`).
- Gerenciador de pacotes **npm** na versão 11 ou superior.
- Navegador web moderno e atualizado (Google Chrome, Firefox, Edge, Safari).
- **Git** instalado para realizar a clonagem do repositório.
- Ferramenta externa ou extensão de IDE para exportar os documentos textuais *Markdown* (.md) para PDF.

Principais Tecnologias e Bibliotecas do Protótipo:

- **Ferramentas de Desenvolvimento e Build:** `vite`, `concurrently`.
- **Frontend:** `react`, `react-dom`, `lucide-react` (para renderização de ícones SVG).
- **Backend e API:** `express`, `google-auth-library` (para validação do token OAuth 2.0).
- **Persistência:** Módulo nativo `node:sqlite`.

Serviços Necessários

Serviço de Apresentação (Frontend)

Responsável por entregar os artefatos estáticos (HTML, CSS e pacotes JS). Este serviço renderiza a interface do usuário, processa os eventos do navegador, controla o estado da aplicação no lado do cliente (com React) e estabelece comunicação com o *backend* através de requisições assíncronas (HTTP).

Serviço de Regras de Negócio (API Backend)

Responsável por implementar a lógica sistêmica. Expõe rotas RESTful para operar fluxos de:

- Cadastro e Autenticação (locais e, opcionalmente, integração com Google);
- Projetos e permissões;
- Quadros e instâncias de raias e colunas;
- Manipulação de cartões;
- Registros de movimentações de estado;

- Validação algorítmica de limites WIP;
- Cálculo unificado de métricas.

Serviço de Banco de Dados (Persistência)

Responsável por armazenar de forma segura os dados de usuários, projetos, membros, quadros, colunas, raiais, cartões e seus históricos temporais de movimentações. No contexto estrito do protótipo acadêmico, o sistema utiliza o motor local **SQLite**.

Configuração e Execução

As instruções abaixo devem ser executadas no terminal na raiz da pasta `/prototype`:

1. Instalar as dependências:

```
# Entra no diretório do protótipo e baixa os pacotes necessários
cd prototype
npm install
```

2. Executar o sistema em ambiente de desenvolvimento (Demonstração):

```
# Sobe o banco, o backend e o frontend simultaneamente
npm run dev
```

3. Configuração Opcional para Login via Google OAuth:

Caso o avaliador queira testar a integração, basta exportar as chaves como variáveis de ambiente antes de rodar o servidor:

```
# Em terminais Windows (PowerShell):
$env:VITE_GOOGLE_CLIENT_ID="seu-client-id-google"
$env:GOOGLE_CLIENT_ID="seu-client-id-google"
npm run dev
```

Nota de Resiliência: Sem o fornecimento explícito dessas variáveis de ambiente, o sistema ignora o fluxo externo e permanece 100% funcional utilizando os processos de cadastro e *login* locais.

4. Executar rotinas de validação e testes:

```
# Verifica sintaxe e padronização (linter/types)
npm run check
# Roda os cenários de testes sistêmicos
npm run test:system
```

Requisitos de Implantação em Produção

Para elevar este sistema à categoria de Produto e disponibilizá-lo em produção real, as seguintes arquiteturas e processos seriam obrigatórios:

- Hospedagem profissional dedicada para a aplicação Node.js (ex: Render, Heroku ou AWS EC2).
- Geração otimizada e minificada dos artefatos estáticos do *frontend* (`npm run build`).
- Gerenciamento seguro de chaves sensíveis em cofres de Variáveis de Ambiente e segredos.
- Banco de dados persistente operando de forma autônoma e dissociado do servidor de aplicação.
- Rotinas e *scripts* de *backup* diário do banco de dados.
- Configuração de certificados de segurança TLS/SSL (HTTPS) para tráfego criptografado de senhas e *tokens*.
- Controle e armazenamento centralizado de *logs* de acesso e erros do servidor.
- Monitoramento de disponibilidade e tempo de inatividade (*Uptime*).
- Esteira de integração contínua (CI/CD) para política de atualização, testes e manutenção evolutiva.

Delimitação e Observação de Escopo

No âmbito estrito deste trabalho acadêmico, a implantação completa e em larga escala de um ambiente de produção **não é o foco principal**. O protótipo tem como objetivo primário ser **executável e avaliável no ambiente local do revisor**, servindo exclusivamente como prova de conceito (*Proof of Concept*) de que a Engenharia de Requisitos, a Arquitetura Projetada e o Modelo de Dados Físico foram integrados e implementados com sucesso e coesão.